YouthPath 시스템 동작 프로세스

YouthPath 시스템 동작 프로세스

> 사용자 자연어 질의 1번으로 청년정책·채용공고·자소서 작성 프롬프트·마감일을 통합 응답하는 Multi-Agent 시스템이 실제로 어떻게 동작하는지 처음부터 끝까지의 흐름.

---

0. 전체 흐름 한 장

```
[사용자 입력]
   ↓
[FastAPI 백엔드] ——— DynamoDB (프로필·대화기록)
   ↓
[LangGraph Router]
   ├─ ① 분류 LLM (LUXIA)
   ├─ ② Policy Agent    ┐
   ├─ ② Job Agent       ├─ 병렬 실행
   ├─ ② Coach Agent     │ (Coach만 LLM 1번 추가)
   ├─ ② Calendar Agent  ┘
   ├─ ③ 포맷터 (JSON → 자연어 텍스트)
   └─ ④ 통합 LLM (LUXIA)
   ↓
[Streamlit Frontend] — 답변 텍스트 + 5탭 카드
```

LLM 호출 횟수: 기본 2번 (분류 + 통합). Coach Agent가 호출된 경우만 +1번 → 최대 3번. Policy / Job / Calendar는 LLM을 전혀 안 씀 (코드·API·룰·점수·벡터검색).

---

1. 예시 시나리오로 따라가기

사용자 질의:

> "서울에서 월세 도와주는 정책이랑 IT 신입 공고 알려줘"

사용자 프로필 (사이드바에서 1번만 입력):

```
나이 27, 서울, 중위소득 60% 이하, Python/SQL, 데이터 분석가, 관심 기업: 네이버
```

---

2. STEP 1 — 입력이 백엔드로 전달

Streamlit이 FastAPI에 HTTP POST를 보냄:

```
POST /ask
{
  "user_id": "demo_001",
  "query": "서울에서 월세 도와주는 정책이랑 IT 신입 공고 알려줘",
  "profile": {
    "age": 27,
    "region": "서울",
    "skills": ["Python", "SQL"],
    "target_role": "데이터 분석가",
    "target_company": "네이버"
  }
}
```

FastAPI가 받자마자 다음을 수행:

1. DynamoDB `UserProfiles` 에서 기존 프로필 조회 (있으면 합침)
2. DynamoDB `Conversations` 에 "user 메시지" 저장
3. LangGraph Router 그래프 호출

---

3. STEP 2 — 분류 LLM (LLM 호출 1번째)

LUXIA에 짧은 분류용 프롬프트 1개를 던짐. 이때 **질의뿐 아니라 프로필 요약 4개 필드**(`region` , `target_role` , `target_company` , `age`)를 같이 넘김 — 모호한 질의에서 의도 파악 정확도를 올리기 위해서. (전체 프로필을 다 넘기지 않는 이유: 분류는 짧을수록 빠르고, 불필요한 필드는 LLM이 묻지도 않은 에이전트를 끌어다 붙이는 노이즈가 됨)

```
사용자 질의를 분석하여 호출할 에이전트를 모두 선택하세요.

에이전트:
- policy: 청년 정책, 지원금, 복지
- job: 채용공고, 직무, 회사 추천
- coach: 자소서 작성용 프롬프트 생성
- calendar: 마감일, 일정

JSON 형식: {"agents": [...], "reasoning": "..."}

[사용자 프로필 요약]
27세, 서울, 데이터 분석가 지망, 관심 기업 네이버

[사용자 질의]
서울에서 월세 도와주는 정책이랑 IT 신입 공고 알려줘
```

LUXIA 응답:

```
{ "agents": ["policy", "job"], "reasoning": "정책+채용 복합 질의" }
```

→ "policy, job 두 개를 부르자"가 결정됨. (coach, calendar는 이번엔 skip)

프로필 요약이 분류에 효과를 보이는 케이스:

- 질의 `"공고 추천해줘"` → 프로필의 `target_role=데이터 분석가` 덕분에 `job` 에이전트 호출 확정
- 질의 `"내가 받을 수 있는 거 정리해줘"` → 프로필의 `age=27, region=서울` 덕분에 `policy` 호출 판단
- 질의 `"네이버 자소서 어떻게 써?"` → 프로필의 `target_company=네이버` 덕분에 `coach` 호출

> 통합 LLM(STEP 6)에는 **전체 프로필**을 그대로 넘김. 분류 단계는 압축 요약, 답변 생성 단계는 풀 컨텍스트.

---

4. STEP 3 — 각 Agent 병렬 실행 (LLM 안 씀)

4개 Agent는 **LLM을 절대 안 쓰고** 각자 자기 방식으로 데이터를 모음. LangGraph가 `asyncio.gather`로 동시 실행.

4-A. Policy Agent

```
1. Chroma `policies` 벡터 컬렉션에서 의미 검색
   query="월세 지원", filter={"region": "서울"}, k=10
   → 후보 정책 청크 10개

2. 정책 단위로 그룹화
  같은 정책의 청크들을 묶어서 후보 정책 객체 생성

3. 자격 매칭 룰 (LLM 없이 순수 코드 수식)
  각 후보 정책마다:
      score = 0
      if 정책.min_age <= 27 <= 정책.max_age:  score += 0.4
      if "서울" in 정책.region:                 score += 0.3
      if 사용자.소득 ∈ 정책.소득기준:           score += 0.3

4. score >= 0.7 인 정책만 남김
   matched_criteria / unmatched_criteria 필드 채움
```

출력 items 예시:

```
[
  {
    "title": "청년월세지원",
    "deadline": "2026-05-31",
    "matched_criteria": ["나이 27세 ✓", "서울 ✓", "중위 60% ✓"],
    "unmatched_criteria": [],
    "link": "https://..."
  },
  { "title": "주거안정월세대출", "...": "..." }
]
```

4-B. Job Agent

워크넷 5종 OpenAPI를 역할별로 분리해 조합. **사람인은 미보유, 잡코리아는 신청 중** — 통과 시 채용정보 결과에 머지. (자세한 스키마는 JobAgent_워크넷_스키마.md 참조)

```
[A] 텍스트 → 코드 변환 (메모리 캐시 룩업, 외부 호출 0번)
  공통코드:  "서울"      → regionCd "11"
  공통코드:  "신입"      → empTpCd  "1"
  공통코드:  "대졸"      → eduLvCd  "03"
  직업정보:  "데이터분석" → jobCd    "2236"

[B] 채용정보 API 메인 검색 (외부 호출 1번)
  워크넷 채용정보 ?regionCd=11&jobCd=2236&empTpCd=1&...
    → 후보 raw 공고 N개

[C] 강소기업 캐시 룩업 (외부 호출 0번, 부팅 시 미리 로드된 set)
    job.is_strong_sme = (job.bizNo in strong_sme_set)

[D] 직무정보 배치 보강 (24h TTL 캐시, miss 시 외부 호출)
    unique 직무 코드만 추출 → 직무정보 API 호출
    → required_skills, preferred_skills 채움

[E] 적합도 점수 계산 (LLM 없이 수식)
    score = 0.4 * 스킬_매칭률
          + 0.3 * 경력_매칭
          + 0.2 * 지역_매칭
          + 0.1 * 마감_긴급도
          + 0.05 * 강소기업 가산점

[F] 점수 내림차순 정렬 → 상위 5개
```

API 실패 대응:

- 5초 timeout + 3회 지수 백오프 재시도
- Circuit Breaker: 연속 5회 실패 시 60초 차단
- 부분 실패 허용: 직무정보·강소기업만 죽으면 해당 필드 비우고 `metadata.partial=true`
- 하드 실패: 채용정보 자체가 죽으면 빈 items + `error` 필드 채움

출력 items 풀 스키마 (UnifiedJob — 1건):

```json
{
  "wantedAuthNo":       "K162345789",
  "title":              "데이터분석 신입",
  "company":            "○○회사",
  "company_bizno":      "123-45-67890",
  "is_strong_sme":      true,
  "location":           "서울 강남구",
  "region_code":        "11",
  "job_code":           "2236",
  "deadline":           "2026-05-20",
  "days_remaining":     8,
  "posted_at":          "2026-05-08",
  "career_required":    "신입",
  "education_required": "대졸 이상",
  "salary": { "type": "연봉", "value": 3500, "unit": "만원" },
  "required_skills":    ["Python", "SQL", "통계 기초"],
  "preferred_skills":   ["Tableau", "추천 시스템"],
  "fit_score":          0.87,
  "fit_breakdown": {
    "skill": 0.36, "career": 0.30, "region": 0.20,
    "urgency": 0.06, "sme_bonus": 0.05
  },
  "source":             "worknet",
  "url":                "https://www.work.go.kr/..."
}
```

필드 그룹별 출처

그룹	필드	출처
식별	`wantedAuthNo`, `title`, `company`, `company_bizno`	채용정보
위치·시간	`location`, `region_code`, `deadline`, `days_remaining`, `posted_at`	채용정보 + 계산
자격	`career_required`, `education_required`	채용정보
보상	`salary`	채용정보
직무 보강	`job_code`, `required_skills`, `preferred_skills`	직업정보 + 직무정보
회사 보강	`is_strong_sme`	강소기업 (캐시)
점수	`fit_score`, `fit_breakdown`	자체 계산
메타	`source`, `url`	자체 합성

4-C. Coach Agent

이번 질의에선 분류 LLM이 호출 안 했지만, 호출됐다면 다음처럼 동작. **자소서를 직접 써주지 않고**, 사용자가 ChatGPT/Claude에 그대로 붙여 넣어서 자기 자소서를 만들 수 있는 **맞춤형 작성 프롬프트**를 생성.

```
1. 화이트리스트 체크
   profile.target_company("네이버")가 사전 인덱싱된 100개 화이트리스트에 있는지 확인
   → 없으면 즉시 fallback: "이 기업은 인덱싱되지 않았습니다" 메시지로 종료

2. 회사 컨텍스트 수집 (병렬)
   ├─ Chroma `companies` 검색 (Static, DART 사업보고서 + 인재상)
   │    query="사업 내용 R&D 인재상", filter={company:"네이버"}, k=15
   ├─ 네이버 검색 API → 뉴스 (Dynamic, 최근 30일)
   │    query="네이버 신규 사업", display=5
   │    → 무료 (일 25,000회 한도)
   └─ 자소서 문항 JSON (Static, company_questions.json)
        → {"네이버": ["지원 동기", "경험", "비전", "강점"]}

3. 직무 컨텍스트 수집 (이미 호출된 Job Agent 결과 재활용)
   target_company == job_item.company 인 공고가 있으면
     → 해당 JD 텍스트에서 요구 스킬·우대사항 추출
  없으면 → 워크넷/사람인 API로 단발 조회

4. 사용자 컨텍스트
   profile에서 skills, 경력, target_role, 관심사 추출

5. 프롬프트 합성 (LLM 1번 호출)
  위 4개를 합쳐 사용자가 ChatGPT/Claude에 그대로 복붙할 수 있는
  마크다운 형식의 작성 프롬프트를 LUXIA가 생성
    - 페르소나 지정 ("당신은 27세 데이터 분석가 지망생...")
    - 회사 컨텍스트 요약 (사업 + 최근 뉴스 + 인재상)
    - 직무 요구사항 (JD 기반)
    - 자소서 문항별 작성 지시 (4문항)
    - "본인의 실제 경험을 바탕으로 답하세요" 명시 → AI 대필 방지
```

출력 items (1개 회사당 1개):


```json
{
  "company": "네이버",
  "target_role": "데이터 분석가",
  "company_summary": {
    "business_keywords": ["검색", "AI", "추천 시스템", "클라우드"],
    "recent_news": [
      {"title": "네이버, 하이퍼클로바X 기업용 API 확대", "date": "2026-05-08"}
    ],
    "values": ["기술 우선", "글로벌 도전", "데이터 기반"]
  },
  "questions": ["지원 동기", "도전 경험", "5년 후 비전", "강점/약점"],
  "generated_prompt": "당신은 네이버 데이터분석 신입에 지원하는 27세 지원자입니다.\n\n[회사 컨텍스트]\n- 사업: 검색·AI·추천 시스템·클라우드. 최근 하이퍼클로바X 기업용 API를 확대 중\n- 인재상: 기술 우선, 글로벌 도전, 데이터 기반 의사결정\n\n[직무 요구사항]\n- 필수: Python, SQL, 통계 기초\n- 우대: 추천/검색 도메인 경험, 대규모 데이터 처리\n\n[당신의 보유 역량]\n- Python, SQL\n- 학부 프로젝트: RAG + FastAPI 기반 정보검색 시스템\n\n다음 4문항에 대해 본인의 실제 경험을 바탕으로 600자 이내로 답해주세요.\n각 답에는 회사의 사업 맥락 또는 최근 동향과 본인 경험을 1개 이상 연결해주세요.\n\n1. 지원 동기 — ...\n2. 도전 경험 — ...\n3. 5년 후 비전 — ...\n4. 강점/약점 — ...\n\n주의: 사실 관계는 본인 경험 그대로 쓰되, 회사 사업 맥락과의 연결고리를 명시적으로 드러내주세요.",
  "copy_button": true
}
```

핵심: 자소서를 **써주지 않음**. 사용자가 ChatGPT/Claude 같은 외부 LLM에 그대로 붙여 넣어 **자기 경험으로 직접 답하게** 하는 작성 프롬프트만 제공. AI 대필 우려를 피하면서 회사 맞춤 가이드는 제공하는 절충안.

4-D. Calendar Agent

다른 Agent와 다르게 **"수집"이 아니라 "정리"하는 Agent**.

```
1. DynamoDB `UserEvents` 테이블 조회
   key: user_id="demo_001"
   → 사용자가 과거에 저장해둔 이벤트 목록 가져옴

2. 다른 Agent 결과에서 마감일 자동 흡수
   - Policy 결과의 각 item.deadline
   - Job 결과의 각 item.deadline
   → 정규식으로 날짜 추출 (서비스 함수 extract_deadline())
   → 새 이벤트로 DynamoDB UserEvents에 저장

3. 마감일 필터링
   compute_days_remaining(deadline) < 0 인 건 제외 (이미 지난 일정)

4. D-day 계산
  각 이벤트에 days_remaining 필드 추가

5. 정렬
   D-day 짧은 순으로 sort
```

출력 items 예시:

```
[
  {
    "title": "○○회사 데이터분석 신입 마감",
    "deadline": "2026-05-20",
    "days_remaining": 8,
    "link": "https://..."
  },
  {
    "title": "청년월세지원 마감",
    "deadline": "2026-05-31",
    "days_remaining": 19,
    "link": "https://..."
  }
]
```

4개 Agent 동작 방식 비교

Agent	데이터 출처	핵심 알고리즘	LLM
Policy	Chroma `policies` (Static RAG)	자격 매칭 룰 (나이/지역/소득 점수)	❌
Job	Worknet/Saramin API (Dynamic, 실시간)	적합도 점수 (스킬·경력·지역·마감 가중합)	❌
Coach	Chroma `companies` (DART+인재상) + 네이버 검색 API (뉴스) + JD + 자소서 문항 JSON	4개 컨텍스트 합성 → 자소서 작성 프롬프트 생성	✅ (1번)
Calendar	DynamoDB `UserEvents` + 다른 Agent 결과	정규식·날짜 계산·D-day 정렬	❌

---

5. STEP 4 — 각 Agent의 응답 JSON 형식 (도메인마다 다름)

모든 Agent는 **같은 외피**로 응답하되, `items` 안의 키는 도메인마다 다름:

공통 외피 (BaseAgent가 강제)

```
{
  "agent_name": "policy",        # | "job" | "coach" | "calendar"
  "items": [ ... ],              # ← 이 안의 키는 도메인마다 다름
  "sources": [ ... ],
  "metadata": { "latency_ms": 320 },
  "error": null
}
```

Policy items 키

```
{ "title", "deadline", "matched_criteria", "unmatched_criteria", "link" }
```

Job items 키 (UnifiedJob)

```
{
  "wantedAuthNo", "title", "company", "company_bizno", "is_strong_sme",
  "location", "region_code", "job_code",
  "deadline", "days_remaining", "posted_at",
  "career_required", "education_required",
  "salary": { "type", "value", "unit" },
  "required_skills", "preferred_skills",
  "fit_score", "fit_breakdown": { "skill", "career", "region", "urgency",
        "sme_bonus" },
  "source", "url"
}
```

> 풀 스키마와 필드 의미는 JobAgent_워크넷_스키마.md §4 참조.

Coach items 키

```
{ "company", "target_role", "company_summary",
  "questions", "generated_prompt", "copy_button" }
```

Calendar items 키

```
{ "title", "deadline", "days_remaining", "link" }
```

왜 통일 안 하나?

정책엔 "자격 충족 조건"이 핵심이지만 채용엔 "적합도 점수", 자소서 코치엔 "생성된 프롬프트", 일정엔 "D-day"가 핵심임. 억지로 키를 통일하면 의미가 사라져서 일부러 도메인별로 둠.

대신 **외피(`agent_name`, `sources`, `error`)는 통일**해서 Router가 어떤 Agent든 똑같은 방식으로 다룰 수 있게 함.

---

6. STEP 5 — 통합 LLM에 넣기 전, JSON을 자연어로 변환 (포맷터)

여기가 **"JSON 형식이 다른데 어떻게 합치냐"의 답**.

왜 JSON을 그대로 LLM에 안 넣나

1. 중괄호·따옴표·영어 키 이름이 **토큰 낭비**
2. LLM이 영어 키를 무시하거나 잘못 해석하기 쉬움
3. LLM이 빈 영역을 **망상으로 채울 위험**

해결: Agent별 포맷터 함수

Router에 도메인별 포맷터 함수가 있어서 JSON → 한국어 텍스트로 변환.

Policy 포맷터 결과:

```
1. 청년월세지원
   - 마감: 2026-05-31
   - 자격 충족: 나이 27세, 서울, 중위 60%
   - 부족 요건: 없음
   - 신청: https://...
2. 주거안정월세대출
   - ...
```

Job 포맷터 결과:

```
1. ○○회사 – 데이터분석 신입
   – 지역: 서울 강남, 마감: 2026-05-20
   – 적합도: 87%
   – 공고: https://...
```

Coach 포맷터 결과:

```
대상 기업: 네이버 / 직무: 데이터 분석가
회사 컨텍스트: 검색·AI·추천·클라우드 / 최근 하이퍼클로바X API 확대
인재상: 기술 우선, 글로벌 도전
자소서 문항: 4개 (지원동기·도전경험·5년비전·강점약점)
→ 사용자가 ChatGPT/Claude에 그대로 붙여 넣을 수 있는 작성 프롬프트가 생성되었음
   (실제 프롬프트는 카드의 [📋 복사] 버튼으로 가져갈 수 있음)
```

Calendar 포맷터 결과:

```
– ○○회사 데이터분석 신입 마감: D-8 (마감 2026-05-20)
– 청년월세지원 마감: D-19 (마감 2026-05-31)
```

호출 안 된 Agent는 `(해당 결과 없음)` 한 줄로 처리.

---

7. STEP 6 — 통합 LLM (LLM 호출 2번째)

위에서 포맷팅한 텍스트들을 **하나의 큰 프롬프트 슬롯**에 끼워서 LUXIA에 1번 호출.

LUXIA가 받는 최종 프롬프트

```
당신은 청년 사회진입 어시스턴트입니다.
사용자 질문과 아래 전문 에이전트들의 결과를 바탕으로
자연스러운 한국어 답변을 작성하세요.

[사용자 질문]
서울에서 월세 도와주는 정책이랑 IT 신입 공고 알려줘

[사용자 프로필]
27세, 서울, 중위 60%, Python/SQL, 데이터 분석가

[정책 결과]
1. 청년월세지원
   - 마감: 2026-05-31
   - 자격 충족: 나이 27세, 서울, 중위 60%
   - 부족 요건: 없음
   - 신청: https://...
2. 주거안정월세대출
   - ...

[채용 결과]
1. ○○회사 - 데이터분석 신입
   - 지역: 서울 강남, 마감: 2026-05-20
   - 적합도: 87%
   - 공고: https://...

[자소서 코치 결과]
(해당 결과 없음)

[일정 결과]
(해당 결과 없음)

답변 작성 규칙:
1. 친근하지만 전문적인 어조
2. 각 항목 끝에 출처 인용
3. 카드/표가 아닌 자연스러운 문단
4. 사용자가 행동할 구체적 다음 단계 제시
5. "(해당 결과 없음)" 영역은 언급하지 말 것
6. 위에 없는 정보는 절대 만들지 말 것 (할루시네이션 금지)
```

LUXIA 응답

```
서울 거주 27세 기준으로 신청 가능한 월세 지원 2건과
데이터 분석 신입 공고 5건을 정리했어요.

청년월세지원은 자격 요건 모두 충족돼서 가장 유리합니다.
신청 마감이 5월 31일까지라 D-19 안에 신청하셔야 해요.
[출처: 온통청년 API]

채용은 ○○회사 데이터분석 신입이 본인 Python·SQL 기술과
적합도 87%로 가장 잘 맞습니다. 마감이 5월 20일로 임박해요.
[출처: 워크넷·사람인]

다음 단계:
1. 청년월세지원 신청 →
2. ○○회사 지원 →
```

---

8. STEP 7 — 응답 반환 + 화면 렌더링

FastAPI 응답:

```json
{
  "answer": "LUXIA가 만든 자연어 답변 ↑",
  "policy":   { "items": [...] },
  "job":      { "items": [...] },
  "coach":    null,
  "calendar": null
}
```

동시에 DynamoDB `Conversations`에 assistant 메시지 저장 + 발견된 마감일을 `UserEvents`에 자동 저장.

Streamlit 화면: 두 층으로 표시 (탭 없이 한 페이지로)

```
┌──────────────────────────────────┐
│ 💬 답변 (LUXIA 자연어)              │ ← 위층: 흐름·추천 이유
│                                  │
│ 청년월세지원이 자격 모두 충족해서    │
│ 가장 유리합니다. 마감 D-19...       │
│ 채용은 ○○회사가 적합도 87%로 1위... │
└──────────────────────────────────┘

📋 정책 결과
│  ┌──────────────────────────┐
│  │ 청년월세지원                 │  ← 카드 1
│  │ 📅 마감: 2026-05-31          │
│  │ ✓ 충족: 나이, 서울, 중위 60% │
│  │ ✗ 미충족: 없음               │
│  │ [해당 사이트로 가기 →] [📅 달력에 추가하기] │
│  └──────────────────────────┘
│
│  ┌──────────────────────────┐
│  │ 주거안정월세대출             │  ← 카드 2
│  │ ...                          │
│  └──────────────────────────┘

💼 채용 결과
│  ┌──────────────────────────┐
│  │ ○○회사 — 데이터분석 신입     │
│  │ 적합도 87% · 서울 강남 · 마감 05-20 │
│  │ [해당 사이트로 가기 →] [📅 달력에 추가하기] │
│  └──────────────────────────┘

🎯 자소서 코치 (작성 프롬프트)
│  ┌──────────────────────────┐
│  │ 네이버 데이터분석 신입       │
│  │ 회사 컨텍스트 + 직무 요구 + 4개 문항 │
│  │ 작성 프롬프트가 합성됨       │
│  │ [📋 프롬프트 복사하기]       │
│  └──────────────────────────┘

📅 마감 일정 (D-day 순)
│  ┌──────────────────────────┐
│  │ ○○회사 데이터분석 신입 마감 │
│  │ D-8 · 2026-05-20             │
│  │ [해당 사이트로 가기 →] [📅 달력에 추가하기] │
│  └──────────────────────────┘
```

→ 모든 도메인을 **탭 없이 위에서 아래로 한 번에** 표시. 사용자가 탭 클릭 없이 스크롤로 전체 확인.

> 📊 **평가 탭은 개발자 모드(`?dev=1` 또는 환경변수 `DEV_MODE=true`)에서만** 사이드바 아래쪽에 노출. 일반 사용자에게는 보이지 않음.

"raw JSON 카드"의 의미

각 Agent가 만든 `items: list[dict]` (raw JSON)을 **사용자가 보기 좋게 시각 컴포넌트(카드)로 그려주는 것**.

예를 들어 Policy items가 이렇게 들어오면:

```
{
  "title": "청년월세지원",
  "deadline": "2026-05-31",
  "matched_criteria": ["나이 27세 ✓", "서울 ✓", "중위 60% ✓"],
  "unmatched_criteria": [],
  "link": "https://..."
}
```

Streamlit이 JSON을 그대로 보여주지 않고, **렌더링 함수**(`render_policy_card`)가:

- `title` → 큰 글씨 제목
- `deadline` → 📅 아이콘 + 캡션
- `matched_criteria` → 초록색 ✓ 배지
- `unmatched_criteria` → 노란색 ✗ 배지
- `link` → **[해당 사이트로 가기 →]** 버튼 (`target="_blank"` 로 새 탭 열림)
- 별도 **[📅 달력에 추가하기]** 버튼 → 클릭 시 DynamoDB `UserEvents` 에 `{title, deadline, link, source: "policy"}` 저장 후 마이페이지 달력에 점으로 표시됨

으로 화면에 그려줌.

왜 자연어 답변이랑 카드 둘 다 보여주나

위층 (LUXIA 답변)	아래층 (카드)
이해를 위한 것	**행동**을 위한 것
"이 정책이 왜 너에게 유리한지" 자연어 설명	마감일·점수·링크 등 **정확한 사실**
우선순위·추천 이유	한눈에 비교 가능한 시각화
흐름·문맥	클릭해서 바로 신청

LUXIA는 **사람처럼 풀어 말해주는 역할**, 카드는 **데이터를 한눈에 보고 다음 행동(클릭)으로 넘어가게 하는 역할**.

도메인별 카드 모양

- **정책 카드**: 자격 충족/미충족 배지 + 마감일 + **[해당 사이트로 가기]** + **[📅 달력에 추가하기]**
- **채용 카드**: 회사명 + 적합도 % 메트릭 + 위치 + 마감일 + **[해당 사이트로 가기]** + **[📅 달력에 추가하기]**
- **자소서 코치 카드**: 회사 컨텍스트 요약 + 직무 요구사항 + 자소서 문항 + 합성된 프롬프트 + **[📋 프롬프트 복사하기]** 버튼 (외부 링크 없음 → 달력 버튼 없음)
- **일정 카드**: D-day 큰 메트릭 + 마감일 + **[해당 사이트로 가기]** + **[📅 달력에 추가하기]**

> "달력에 추가하기" 클릭 시 동작: 해당 항목의 `{title, deadline, link, source}` 를 `POST /events` 로 백엔드에 전송 → DynamoDB `UserEvents` 에 upsert → 마이페이지 달력에 즉시 반영. 같은 항목을 두 번 누르면 토글로 제거.

---

9. 데이터 저장 위치

데이터	어디	왜
정책 본문 임베딩	Chroma `policies`	잘 안 변함 — 미리 인덱싱 (Static RAG)
회사 사업보고서·인재상 임베딩	Chroma `companies`	잘 안 변함 — 미리 인덱싱 (Static RAG, Coach Agent용)
기업별 자소서 문항	JSON `company_questions.json`	자체 수집, 정적 캐싱 (Coach Agent용)
채용공고	저장 안 함	시시각각 변함 — 매번 API 호출 (Dynamic)
최근 기업 뉴스	저장 안 함	네이버 검색 API 매번 호출 (Dynamic, 무료 한도 일 25,000건)
사용자 프로필	DynamoDB `UserProfiles`	개인 데이터 영속
대화 기록	DynamoDB `Conversations`	멀티턴 컨텍스트
마감일 일정	DynamoDB `UserEvents`	Calendar Agent + 마이페이지가 사용

데이터 처리 3단 분리 원칙:

- 잘 안 변하는 건 미리 임베딩해서 검색 → **Static RAG**
- 시시각각 변하는 건 캐싱하지 말고 매번 새로 → **Dynamic RAG / API**
- 사용자 개인 데이터는 → **DynamoDB**

---

10. 한 줄 요약 + 핵심 트릭

한 줄 요약

```
입력 → 분류 LLM (Agent 고르기)
      → 선택된 Agent들이 코드로 자기 JSON 만듦 (형식은 도메인마다 다름)
      → 포맷터가 JSON을 사람말 텍스트로 변환
      → 통합 LLM이 그 텍스트들을 합쳐 자연어 응답 생성
      → 화면에 답변 텍스트 + raw JSON 카드로 표시
```

핵심 트릭 3개

1. **외피만 통일** — Agent 응답은 같은 wrapper(`agent_name` / `items` / `sources`), 내부 items는 도메인별 자유
2. **JSON을 LLM에 그대로 안 줌** — 포맷터가 도메인별로 자연어로 미리 풀어줌
3. **빈 결과는 `(해당 결과 없음)` 으로 명시** — LLM이 망상으로 채우는 거 방지 + "위에 없는 건 만들지 말 것" 규칙으로 출처 강제

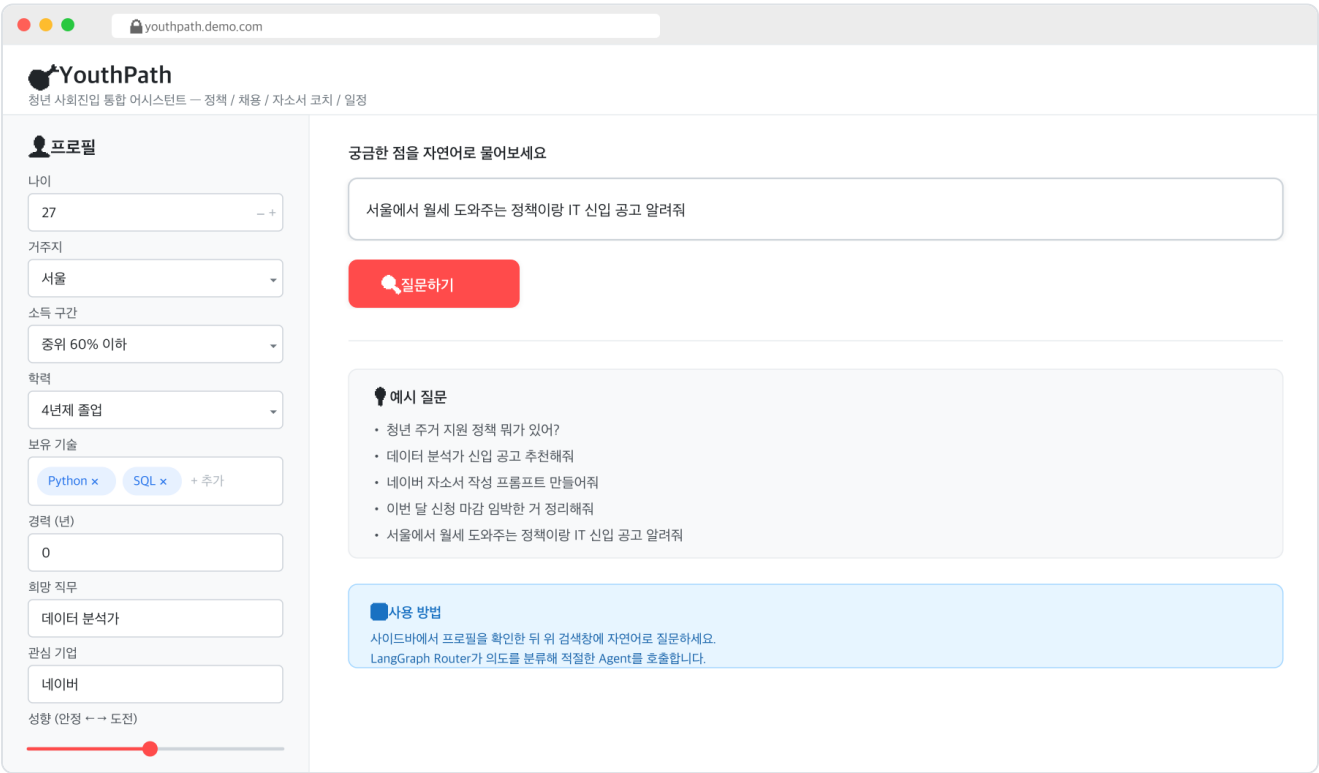
LLM 호출을 최소로 묶은 이유 (기본 2번 · Coach 호출 시 3번)

- 비용 절감 (Agent마다 LLM 부르면 호출 폭증)
- 응답 속도 (LLM 호출이 가장 느린 부분)
- 검증 가능성 (Policy / Job / Calendar 내부는 결정론적이라 테스트 가능)
- 객관성 (LLM은 "이해/표현" 역할, "판단"은 코드)
- **Coach만 예외적으로 LLM 사용**: 4개 소스(회사 컨텍스트·JD·문항·프로필) 합성은 룰베이스로 표현이 어색해서 자연어 합성이 필수. 단 사용자가 Coach를 호출한 경우에만 발생.

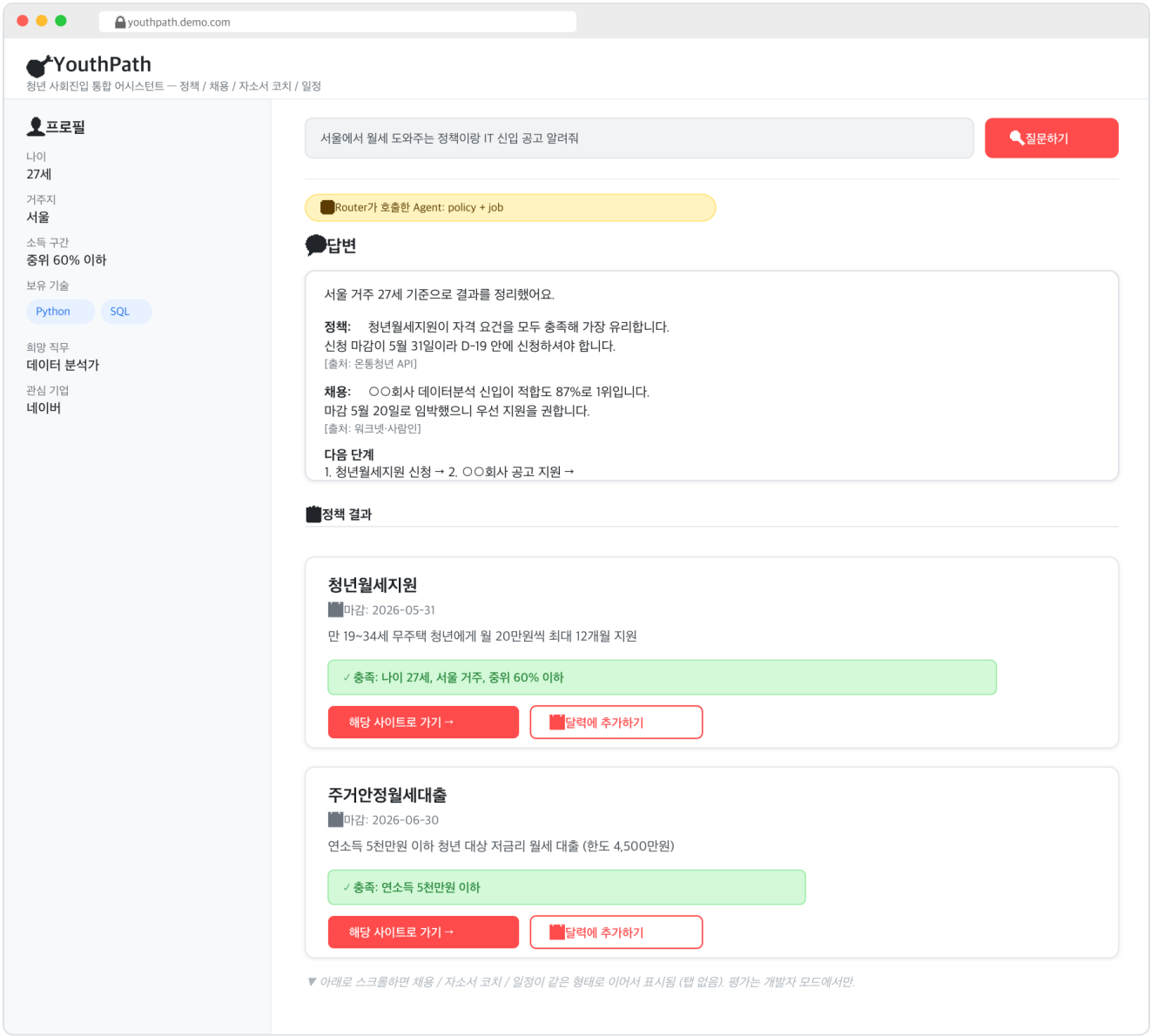
---

11. 화면 배치 (Input / Output)

📥 INPUT 화면



📤 OUTPUT 화면



💼 채용 결과 (정책 아래로 이어서 표시)

```
┌──────────────────────────────────────┐
│ ○○회사 — 데이터분석 신입              │
│ ┌──────┬──────┬──────┐              │
│ │ 적합도 │ 위치  │ 마감  │          │  채용
│ │ 87%  │ 서울 강남 │ 05-20 │          │  카드 1
│ └──────┴──────┴──────┘              │
│ 요구 스킬: Python, SQL, Tableau       │
│ [해당 사이트로 가기 →] [📅 달력에 추가하기] │
└──────────────────────────────────────┘

┌──────────────────────────────────────┐
│ △△기업 — 주니어 데이터 분석가          │
│   적합도 78% · 서울 판교 · 마감 05-25   │
│ [해당 사이트로 가기 →] [📅 달력에 추가하기] │
└──────────────────────────────────────┘
```

🎯 자소서 코치 (채용 아래로 이어서, 작성 프롬프트 생성)

```
┌──────────────────────────────────────┐
│ 📌 네이버 데이터분석 신입 — 작성 프롬프트│
│                                      │
│ 회사 컨텍스트 (DART + 최근 뉴스 + 인재상)│
│   • 검색·AI·추천 시스템·클라우드        │  자소서
│   • 최근: 하이퍼클로바X 기업용 API 확대  │  코치
│   • 인재상: 기술 우선, 글로벌 도전      │  카드
│                                      │
│ 직무 요구사항 (JD 기반)                 │
│   • 필수: Python, SQL, 통계 기초       │
│   • 우대: 추천/검색 도메인, 대규모 데이터│
│                                      │
│ 자소서 문항 (4개): 지원동기 / 도전경험   │
│                  / 5년 비전 / 강점·약점  │
│                                      │
│ — 합성된 프롬프트 미리보기 —            │
│ "당신은 네이버 데이터분석 신입에 지원…   │
│  본인의 실제 경험을 바탕으로 600자…"     │
│                                      │
│ [📋 프롬프트 복사하기]                  │
│ → ChatGPT/Claude에 붙여 넣어 본인 경험  │
│   으로 직접 자소서를 완성하세요         │
└──────────────────────────────────────┘
```

📅 마감 일정 (자소서 코치 아래로 이어서, D-day 짧은 순)

```
┌──────────────────────────────────────┐
│ ○○회사 데이터분석 신입 마감            │
│ ┌──────────┬──────────────┐          │
│ │   D-8    │ 2026-05-20   │          │  일정
│ └──────────┴──────────────┘          │  카드 1
│ [해당 사이트로 가기 →] [📅 달력에 추가하기] │
└──────────────────────────────────────┘

┌──────────────────────────────────────┐
│ 청년월세지원 마감                      │
│   D-19 · 2026-05-31                  │
│ [해당 사이트로 가기 →] [📅 달력에 추가하기] │
└──────────────────────────────────────┘
```

📊 평가 (개발자 모드 전용)

> **일반 사용자에게는 보이지 않음.** URL에 `?dev=1` 을 붙이거나 환경변수 `DEV_MODE=true` 로 실행할 때만 사이드바 맨 아래 "⚙ 개발자 도구" 섹션에 노출. 데모/심사용 화면.

```
📊 시스템 평가 결과 (B1 vs B2 vs B3)

┌──────────────────┬──────┬──────┬──────┐
│ Metric           │ B1   │ B2   │ B3   │
├──────────────────┼──────┼──────┼──────┤
│ Faithfulness     │ 0.42 │ 0.68 │ 0.81 │
│ Context Recall   │ 0.31 │ 0.59 │ 0.74 │
│ Answer Relevance │ 0.55 │ 0.71 │ 0.83 │
└──────────────────┴──────┴──────┴──────┘

▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓ B3 (YouthPath)
▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓         B2 (LLM+RAG)
▓▓▓▓▓▓▓▓▓▓▓▓                 B1 (LLM only)
```

---

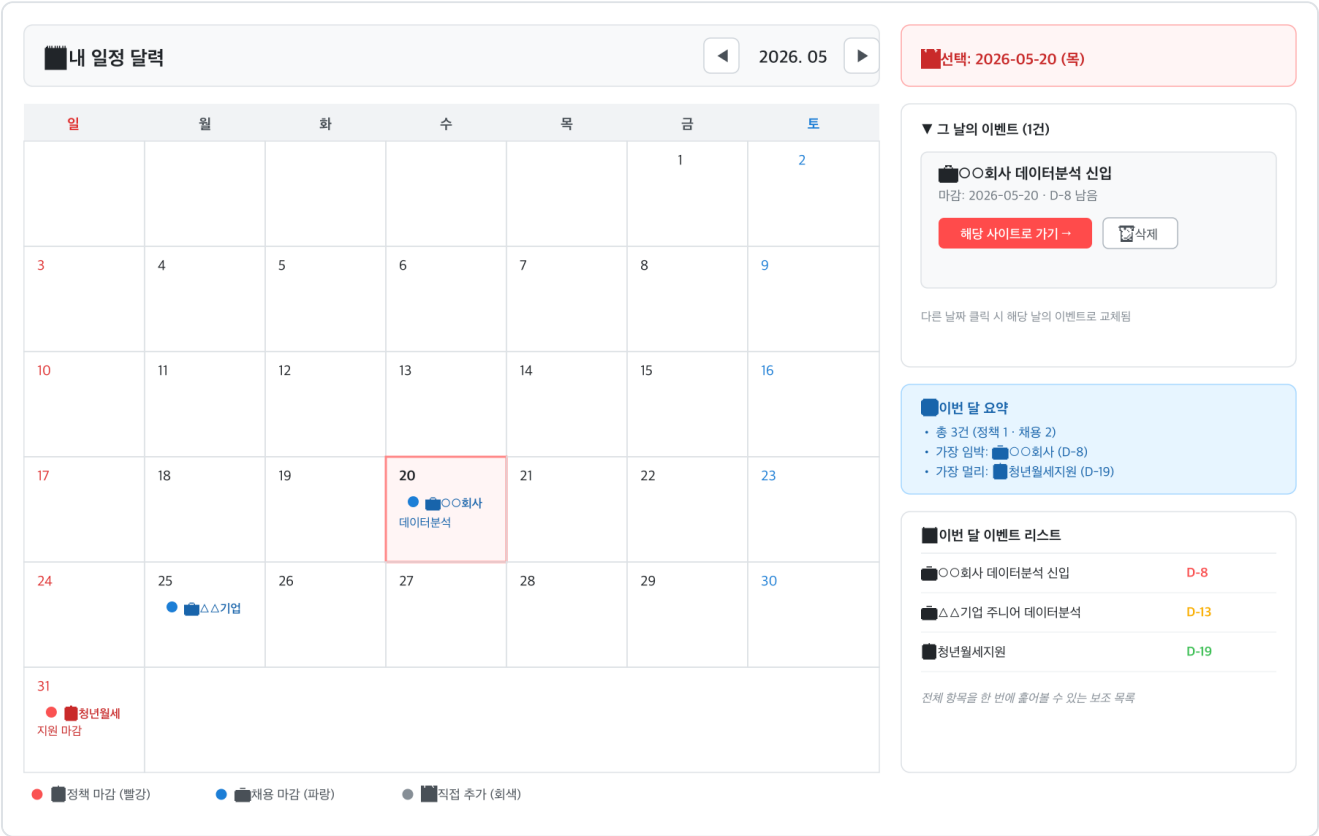
12. 🗓 마이페이지 (내 일정 달력)

상단 헤더의 **[👤 마이페이지]** 버튼으로 이동. 사용자가 각 카드에서 **[📅 달력에 추가하기]**로 저장한 항목들을 **월별 달력 모양**으로 보여주는 페이지.

데이터 흐름

```
[카드의 📅 달력에 추가하기 클릭]
        ↓
POST /events { user_id, title, deadline, link, source }
        ↓
DynamoDB UserEvents 테이블 upsert
        ↓
[마이페이지 진입 시]
        ↓
GET /events?user_id=... &month=2026-05
        ↓
달력 그리드에 마감일별로 점/배지 렌더링
```

화면 레이아웃



> 📋 = 정책 마감 (빨강) · 💼 = 채용 마감 (파랑) · 📅 = 직접 추가 (회색). 날짜 셀 클릭 시 우측 패널에 해당 날의 이벤트 카드가 펼쳐지며 `[해당 사이트로 가기]` · `[🗑 삭제]` 버튼이 노출됨.

마이페이지 기능 요약

기능	동작
월 이동	`◀ ▶` 버튼으로 이전/다음 달, 클릭 시 `GET /events?month=...` 재호출
날짜 셀	해당 날짜의 이벤트 개수만큼 색상 점 표시 (정책 빨강 / 채용 파랑 / 일정 회색)
날짜 클릭	우측 패널에 그 날의 이벤트 카드 목록 펼침
이벤트 카드	제목 + D-day + `[해당 사이트로 가기]` + `[🗑 삭제]`
삭제	`DELETE /events/{event_id}` → DynamoDB에서 제거 → 달력 즉시 갱신
빈 상태	추가된 이벤트가 없으면 "결과 페이지에서 '📅 달력에 추가하기'를 눌러 일정을 모아보세요" 안내

Calendar Agent와의 관계

기존 STEP 4-D의 Calendar Agent는 **답변 화면의 "마감 일정 섹션"용**(자동 흡수된 마감일을 D-day 정렬해서 보여줌). 반면 마이페이지 달력은 **사용자가 명시적으로 [📅 달력에 추가하기] 누른 항목만** 표시 → 둘 다 같은 `UserEvents` 테이블을 쓰지만, 마이페이지는 `is_pinned=true` 필드로 사용자가 직접 담은 것만 필터링.

```
# 답변 화면의 일정 섹션 (Calendar Agent)
items = UserEvents.query(user_id=..., auto_filled=True)  # 자동 흡수분 포함

# 마이페이지 달력
items = UserEvents.query(user_id=..., is_pinned=True)    # 사용자가 담은 것만
```

핵심 배치 요약

- **왼쪽 사이드바** = 프로필 (한 번 입력하면 세션 내내 유지) + 상단에 `[🔲 마이페이지]` 진입 버튼
- **상단 메인** = 질문창 + 질문하기 버튼
- **중단** = 호출된 Agent 표시 + 💬 LUXIA 자연어 답변
- **하단** = 정책 / 채용 / 자소서 코치 / 일정을 **탭 없이 한 페이지에 순서대로** 표시. 외부 링크가 있는 카드(정책·채용·일정)에는 `[해당 사이트로 가기]` + `[📅 달력에 추가하기]`, 자소서 코치 카드에는 `[📋 프롬프트 복사하기]`
- **마이페이지** = 사용자가 담아둔 일정을 **월별 달력**으로 시각화
- **개발자 모드 전용** = 📊 평가 섹션 (`?dev=1` 일 때만 노출)